

The poster features a dark blue night sky background with small white stars. A large, multi-colored rainbow (red, orange, yellow, green, blue, purple) flows from the left side of the frame. In the bottom left corner, there is a white silhouette of a suspension bridge with two green trees on either side. In the bottom right corner, there is a silhouette of a city skyline with several buildings. In the top right corner, there is a red and yellow structure resembling a space station or a large satellite. A small blue and white rocket is flying towards the right. A green vine with a white flower is on the right side. The text 'PyConUS 2025' is written in a large, stylized, white font with a red outline, and 'Pittsburgh' is written in a smaller, white, serif font below it.

PyConUS 2025

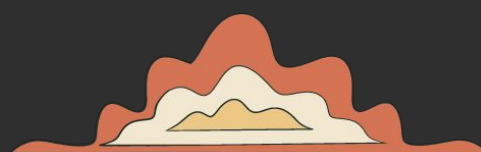
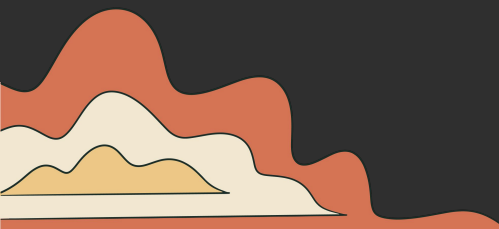
Pittsburgh

Django Asíncrono

The background is a dark blue night sky filled with small white stars. A multi-colored rainbow (red, orange, yellow, green, blue, purple) curves from the left side of the frame. In the bottom left corner, there is a white suspension bridge with two green trees on either side. In the bottom right corner, there is a stylized city skyline with several buildings in white and red. On the right side, there is a white flower with a green vine and a small blue and white rocket ship flying upwards.

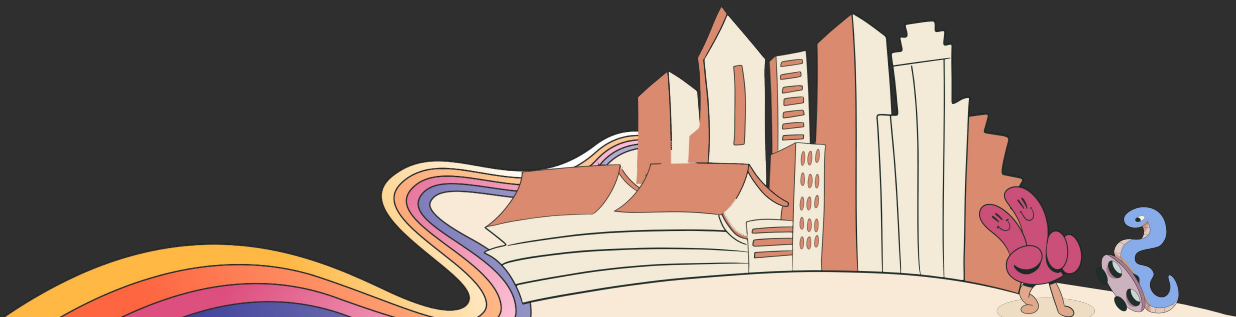
**Desbloqueando el Poder de la
Programación Async**

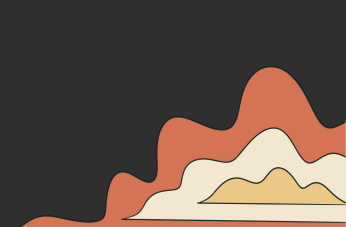
Luis Martinez - lcmartinez.com



El tiempo de espera no es tiempo productivo.

La promesa del asincronismo

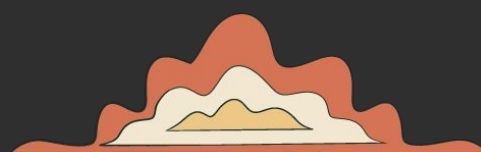
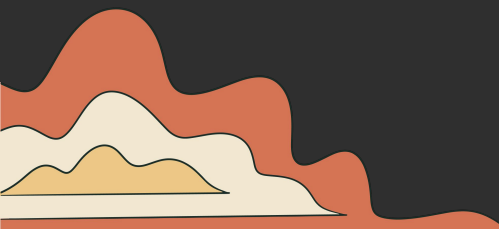




 Lentitud acumulada

 Posibles bloqueos totales

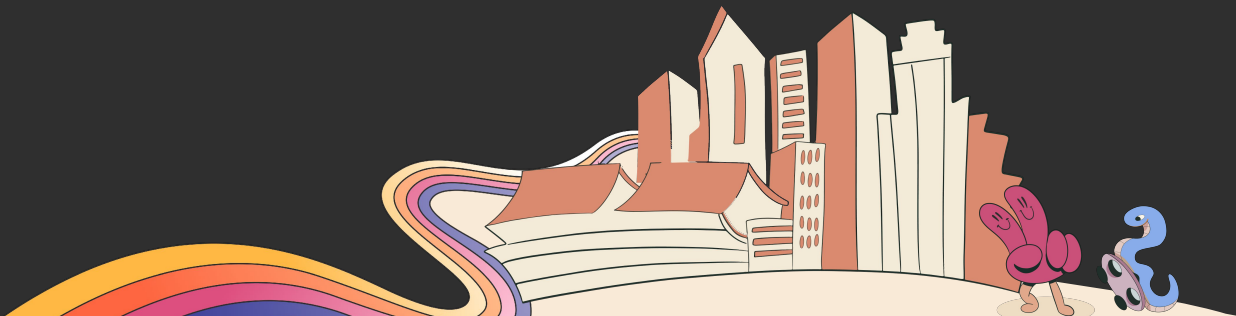
 Poca escalabilidad

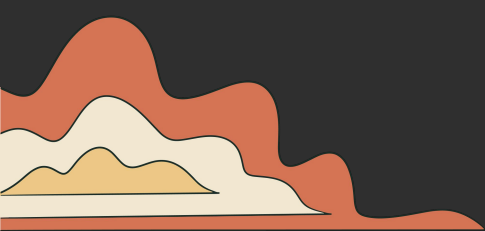


I/O

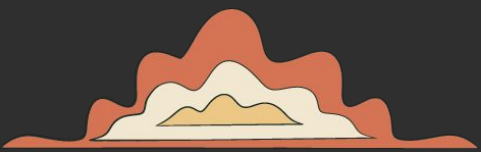
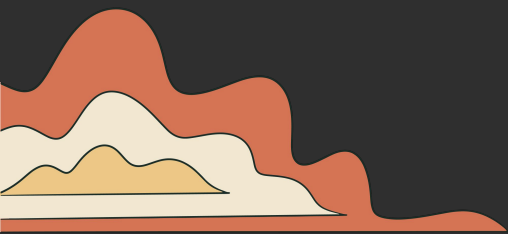
El Gran Bloqueador

APIs externas, base de datos, escritura en disco



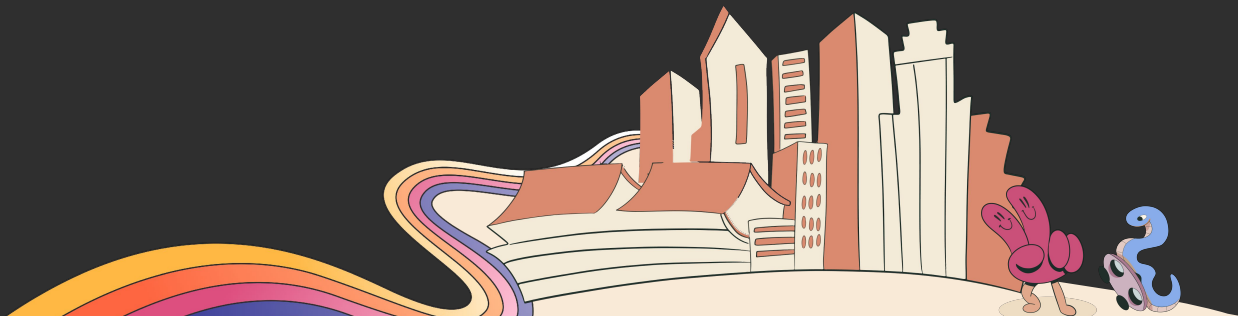


¿Soluciones?



Asyncio

permite ejecutar tareas al mismo tiempo sin esperas





Más peticiones simultáneas

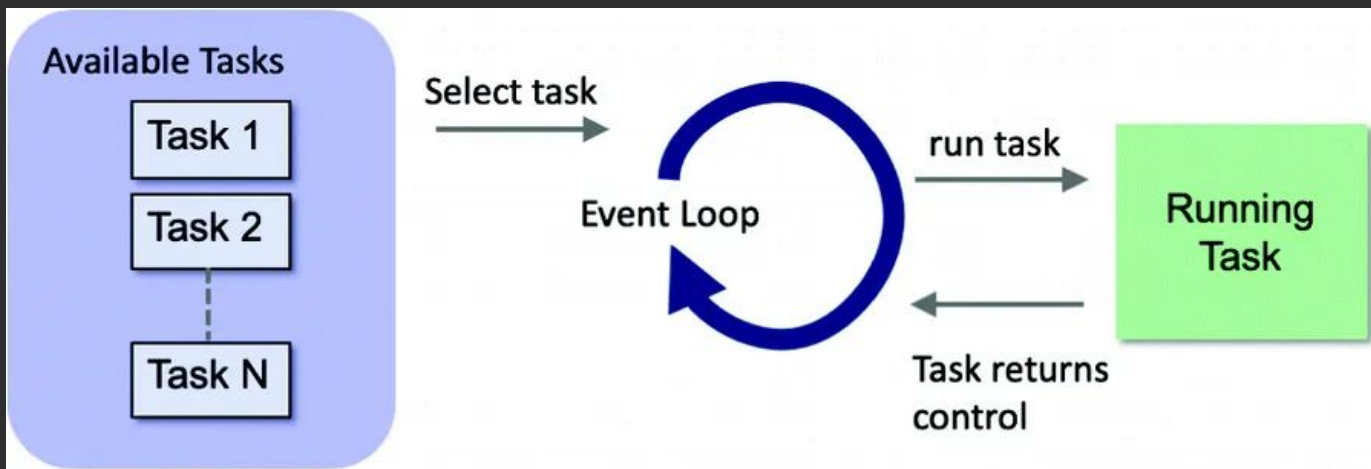


Reducción de tiempos de respuesta



Escalabilidad natural

¿Cómo funciona el Event Loop?



*Correr una webapp con asyncio requiere un server asgi



async / await

Son la base de la programación asíncrona

Sync

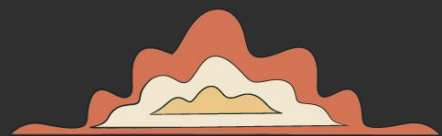
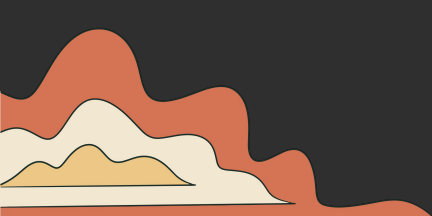


```
1 import time
2
3 def saludar():
4     # Simula esperar por algo (API, DB, etc.)
5     time.sleep(1)
6     return "¡Hola PYCON USA 2025!"
```

Async

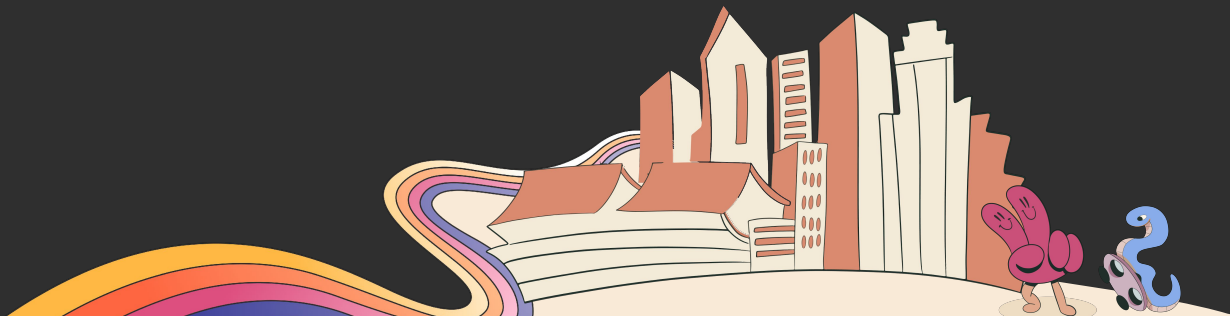


```
1 import asyncio
2
3 async def saludar():
4     # Simula esperar por algo (API, DB, etc.)
5     await asyncio.sleep(1)
6     return "¡Hola PYCON USA 2025!"
```

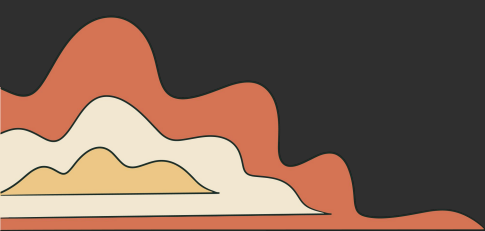


Django + Async

¿Cómo usar asyncio en Django?

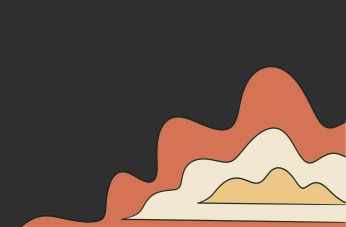






Soporte para ASGI

Nuevas librerías, nuevo archivo .py, nueva app



pip install django

● ● ● requirements.txt

```
1 asgiref==3.8.1
2 Django==5.2.1
3 sqlparse==0.5.3
```



▼ DJANGO-ASYNC-CHARLA

▼ django_async_charla

 `__init__.py`

 `asgi.py`

 `settings.py`

 `urls.py`

 `wsgi.py`

 `.gitignore`

 `manage.py`

asgi.py

```
1 """
2 ASGI config for django_async_charla project.
3
4 It exposes the ASGI callable as a module-level variable named ``application``.
5
6 For more information on this file, see
7 https://docs.djangoproject.com/en/5.2/howto/deployment/asgi/
8 """
9
10 import os
11
12 from django.core.asgi import get_asgi_application
13
14 os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'django_async_charla.settings')
15
16 application = get_asgi_application()
17
```



Versión

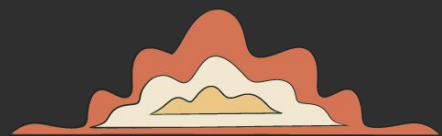
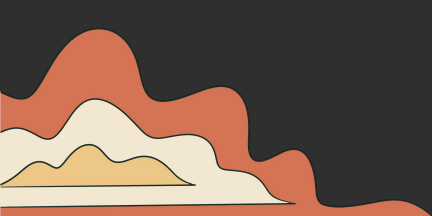


Fecha de Lanzamiento



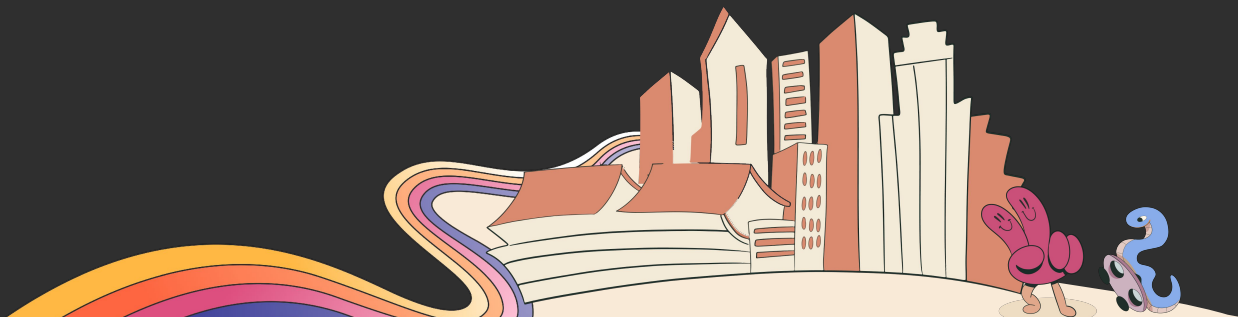
Características Async Principales

3.0	Diciembre 2019	Soporte ASGI (base para async en Django)
3.1	Agosto 2020	Soporte para vistas <code>async def</code>
3.2	Abril 2021	Mejoras internas, preparación para async en el ORM
4.0	Diciembre 2021	Soporte parcial para ORM async (consultas simples)
4.1	Agosto 2022	Soporte async en middlewares y señales
4.2	Abril 2023	Fixtures y pruebas async más robustas
5.0	Diciembre 2023	Mejoras significativas en el ORM async
5.1	Agosto 2024	Soporte para RequestFactory, AsyncRequestFactory, Client, and AsyncClient
5.2	Abril 2025	Soporte para django.contrib.auth



ORM + Async

Django ha ido integrando capacidades asíncronas en su ORM con cada nueva versión.



Nuevos métodos del ORM que interactúan con la base de datos.

```
aget()  
acreate()  
asave()  
adelete()
```

Muchos otros...



```
1 # 📌 ORM síncrono tradicional
2 books = Book.objects.filter(author="Gabo")
3 first_book = books.first()
4
5 # ⚡ ORM asíncrono (Django moderno)
6 books = Book.objects.filter(author="Gabo")
7 first_book = await books.afirst()
```



```
1 # 📌 Iteración síncrona tradicional
2 for book in Book.objects.filter(author="Vargas Llosa"):
3     print(book.title)
4
5 # ⚡ Iteración asíncrona
6 async for book in Book.objects.filter(author="Vargas Llosa"):
7     print(book.title)
8
9
```



Limitaciones del ORM

Todo el ORM aún no es async



Las transacciones NO
funcionan en modo async



```
1 from django.db import transaction
2
3
4 async def broken_transaction():
5     async with transaction.atomic():
6         await model.asave()
7     return "Works?"
```



NO funcionan en modo async

SynchronousOnlyOperation at /broken_transaction/

You cannot call this from an async context - use a thread or sync_to_async.

Request Method: GET
Request URL: http://127.0.0.1:8000/broken_transaction/
Django Version: 5.2.1
Exception Type: SynchronousOnlyOperation
Exception Value: You cannot call this from an async context - use a thread or sync_to_async.
Exception Location: /Users/luismartinez/.envs/django_async_charla/lib/python3.12/site-packages/django/utils/asyncio.py, line 24, in inner
Raised during: demo.views.broken_transaction
Python Executable: /Users/luismartinez/.envs/django_async_charla/bin/python
Python Version: 3.12.8
Python Path: ['/Users/luismartinez/projects/django-async-charla',
'/Users/luismartinez/.pyenv/versions/3.12.8/lib/python312.zip',
'/Users/luismartinez/.pyenv/versions/3.12.8/lib/python3.12',
'/Users/luismartinez/.pyenv/versions/3.12.8/lib/python3.12/lib-dynload',
'/Users/luismartinez/.envs/django_async_charla/lib/python3.12/site-packages']
Server time: Sat, 17 May 2025 16:03:12 +0000

Traceback [Switch to copy-and-paste view](#)

/Users/luismartinez/.envs/django_async_charla/lib/python3.12/site-packages/django/core/handlers/exception.py, line 55, in inner

```
55.         response = get_response(request)
                        ~~~~~
```

► Local vars

/Users/luismartinez/.envs/django_async_charla/lib/python3.12/site-packages/django/core/handlers/base.py, line 197, in _get_response

```
197.         response = wrapped_callback(request, *callback_args, **callback_kwargs)
                        ~~~~~
```

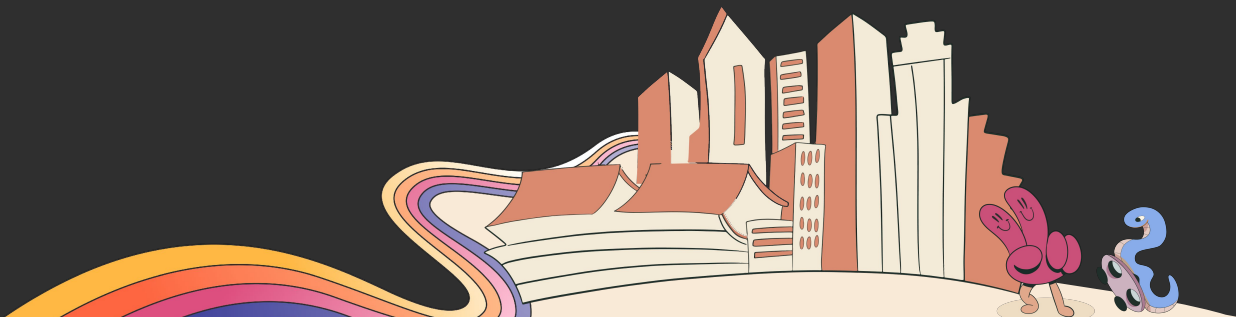
► Local vars

/Users/luismartinez/.pyenv/versions/3.12.8/lib/python3.12/concurrent/futures/_base.py, line 449, in result

```
449.         return self._get_result()
                        ~~~~~
```

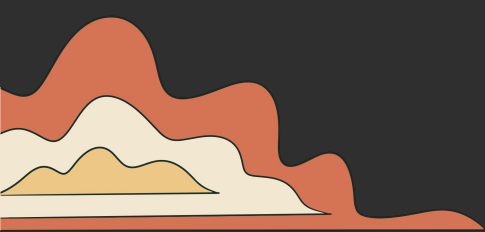
sync_to_async

Ejecuta funciones sync en un hilo, evitando bloquear el event loop.





```
1 from asgiref.sync import sync_to_async
2 from django.db import transaction
3
4
5 @sync_to_async
6 def another_sync_method():
7     with transaction.atomic():
8         model.asave()
9
10 async def broken_transaction():
11     await another_sync_method()
12     return "Works?"
13
```

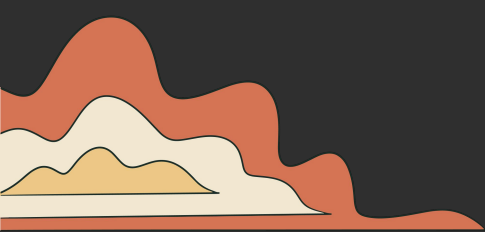


Escribiendo una view async



```
1 import requests
2
3 from django.http import JsonResponse
4
5 base_url = "https://api.weatherapi.com/v1/current.json"
6 london_url = f"{base_url}?key=TBA&q=London"
7 tokyo_url = f"{base_url}?key=TBA&q=Tokyo"
8
9 def weather_view(request):
10     london_weather = requests.get(london_url)
11     tokyo_weather = requests.get(tokyo_url)
12
13     return JsonResponse({
14         "london": london_weather.json(),
15         "tokyo": tokyo_weather.json(),
16     })
17
18
```

```
1 import httpx
2
3 from django.http import JsonResponse
4
5 base_url = "https://api.weatherapi.com/v1/current.json"
6 london_url = f"{base_url}?key=TBA&q=London"
7 tokyo_url = f"{base_url}?key=TBA&q=Tokyo"
8
9 async def weather_view(request):
10     async with httpx.AsyncClient() as client:
11         london_weather = await httpx.get(london_url)
12         tokyo_weather = await httpx.get(tokyo_url)
13
14     return JsonResponse({
15         "london": london_weather.json(),
16         "tokyo": tokyo_weather.json(),
17     })
18
```



DEMO





lcmartinezdev/django-async- x +

github.com/lcmartinezdev/django-async-charla

lcmartinezdev / django-async-charla

Pin Unwatch 1 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file + <> Code

lcmartinezdev Add Readme and requirements 031eac · 2 minutes ago 3 Commits

demo	Add example comparing synchronous and asynch...	10 minutes ago
django_async_charla	Add example comparing synchronous and asynch...	10 minutes ago
.gitignore	Project initialization	2 hours ago
README.md	Add Readme and requirements	2 minutes ago
manage.py	Project initialization	2 hours ago
requirements.txt	Add Readme and requirements	2 minutes ago

README

Django Asíncrono Demo

Este repositorio contiene el código de ejemplo utilizado en la charla "Django Asíncrono: Desbloqueando el Poder de la Programación Async" para PyCon US.

About

Este repositorio contiene el código de ejemplo utilizado en la charla "Django Asíncrono: Desbloqueando el Poder de la Programación Async" para PyCon US.

Readme

Activity

0 stars

1 watching

0 forks

Languages

Python 100.0%

Suggested workflows

Based on your tech stack

Python Package

Configure



Luis Martinez

Cursos impartidos



Curso de Django

★★★★★ 124 opiniones



Curso de Unit Testing en Python

★★★★★ 51 opiniones



Curso de Django Rest Framework

★★★★★ 58 opiniones



Curso de FastAPI

★★★★★ 54 opiniones



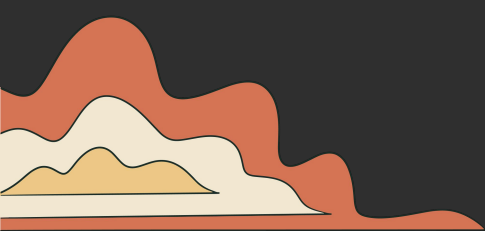
Curso de Despliegue de
Aplicaciones Python en la Nube

★★★★★ 12 opiniones



Curso de Flask

★★★★★ 11 opiniones



Gracias!

lcmartinez.com